Ivent – Event Check-In System

MIC Recruitment 2nd & 3rd Year Task • Production-Ready Offline-First Check-In & Telemetry Engine

1. Short Project Description

Ivent is a high-concurrency, offline-resilient event registration and admission control system built with **Next.js 16.3.1 (React 19.2.8)** on the frontend and an **Express 4.21.2 + PostgreSQL 8.13.1** microservice backend. It features dynamic 30-second rotating TOTP QR codes to eliminate ticket screenshot fraud, atomic capacity locks to prevent race conditions during high-demand registrations, and client-side IndexedDB outbox queuing with idempotent synchronization for seamless zero-internet check-ins. Real-time telemetry is pushed via **Socket.io 4.8.1**, and administrative analytics are powered by context-grounded AI insights with deterministic database computation fallbacks.

2. Features Built & Technical Architecture

CORE 1-3 **Event Creation & Unique QR Tokens**

Organizers create events linked to clubs with capacity constraints. Each registration dynamically generates an RFC 6238 160-bit TOTP secret stored securely per attendee ticket.

CORE 4-5 **Live Camera Scanner & Telemetry**

HTML5 camera stream with auto-facing mode detection processes payloads with instant feedback. Real-time check-in counts are pushed instantly over Socket.io rooms.

CORE 6-7 **Per-Event Roles & Telemetry Export**

Normalized `event_organizers` and `club_members` join tables ensure granular per-event authorization. Verified organizers can export full attendee check-in logs as standard RFC 4180 CSV files.

HARD REQ 1 **Duplicate Prevention & Capacity**

Solved via single-statement atomic SQL updates eliminating race gaps:

```
• UPDATE events SET registered_count = registered_count
+ 1 WHERE id = $1 AND registered_count < capacity
• UPDATE registrations SET checked_in_at = NOW() WHERE
id = $1 AND checked_in_at IS NULL
```

Guarantees zero overbooking and exactly one accepted check-in under high concurrency.

HARD REQ 2 **Anti QR-Sharing (Rotating TOTP)**

Implemented rotating 30-second HMAC-SHA1 TOTP tokens computed via Web Crypto API. Screenshots expire within 30 seconds, and the backend verifies tokens with a ±1 window drift tolerance.

HARD REQ 3 **Offline Scanning & Conflict Sync**

Scans are saved to client-side IndexedDB outbox. When reconnected, batch sync runs idempotently with `client_scan_id` uniqueness in `scan_log`. Multi-station conflicts accept the earliest timestamp and cleanly reject duplicate scans.

HARD REQ 4 **AI Insights (Context-Grounded)**

Natural language queries compute verified SQL metrics (checked-in counts, no-show rates, peak arrival windows, remaining seats) first, feeding them as deterministic context to **Google Gemini API** with offline heuristic fallbacks.

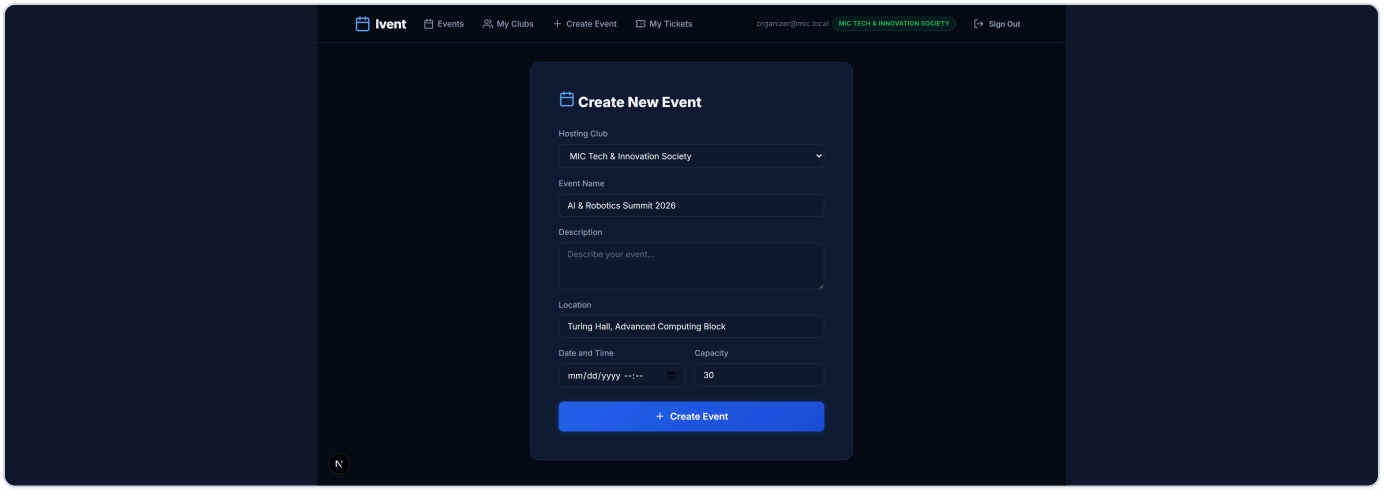
COMPLETED & VERIFIED **System Scope Statement**

All 7 Core Requirements and all 4 Hard Requirements are fully implemented, verified, and backed by automated concurrency and end-to-end test suites. WebSockets (Socket.io) are actively utilized for instant live telemetry across organizer dashboards and event rooms.

3. Application Screenshots & Live Verification

1. Event Creation Form (Organizer View)

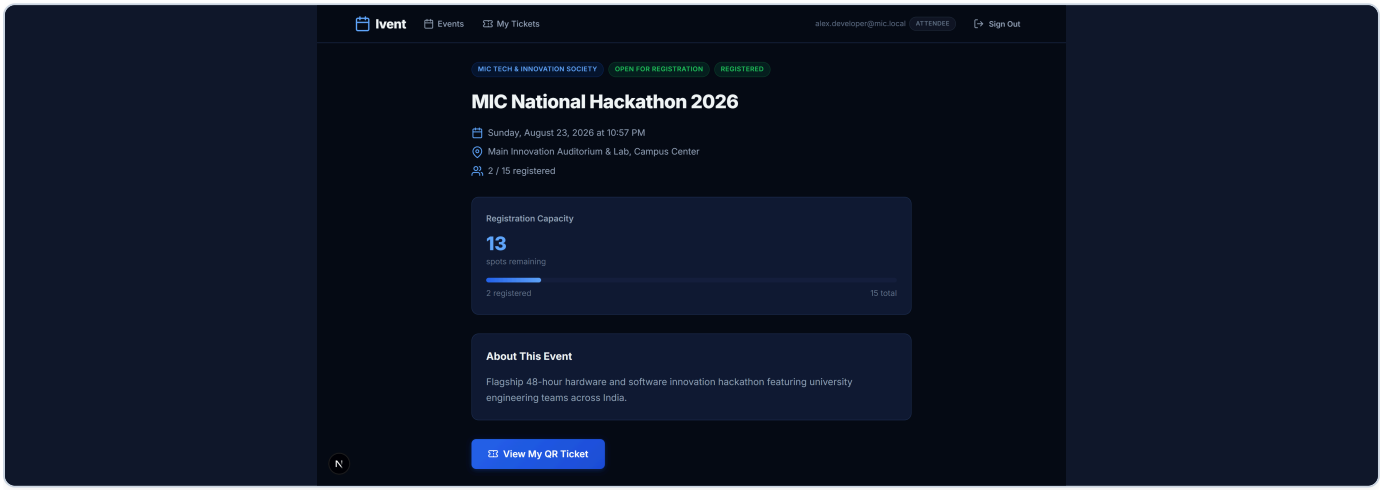
ORGANIZER CONSOLE



Organizers create events linked to their verified club memberships, setting dates, capacity limits, and descriptions with validation.

2. Attendee Registration Flow & Live Event View

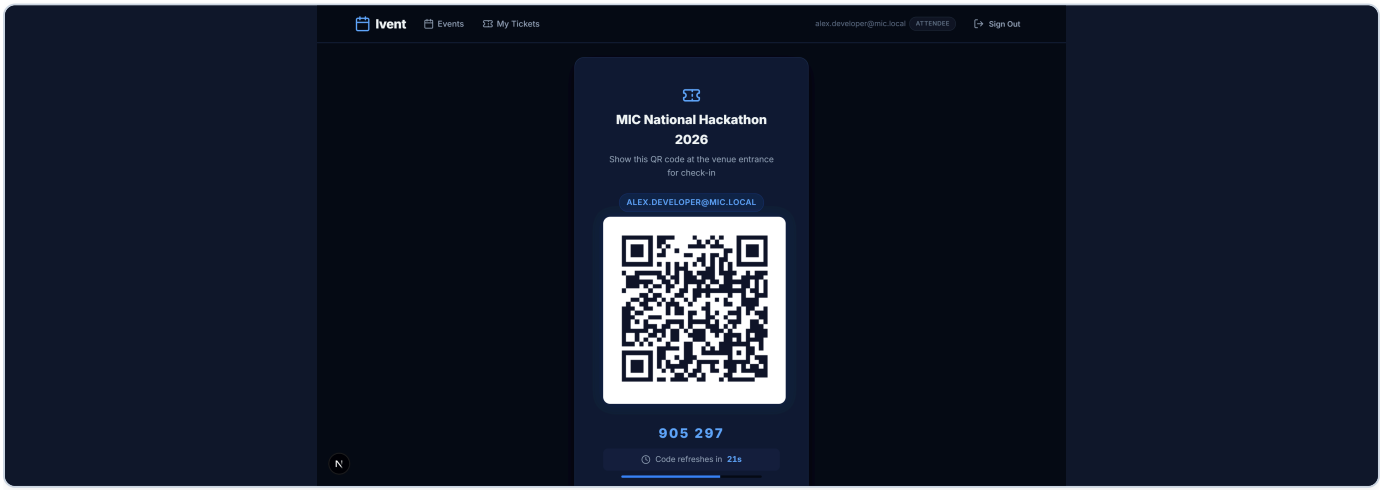
ATTENDEE PORTAL



Clean registration interface showing real-time capacity telemetry and atomic one-click ticket reservation.

3. Dynamic Attendee Ticket (Rotating 30-Second TOTP QR Code)

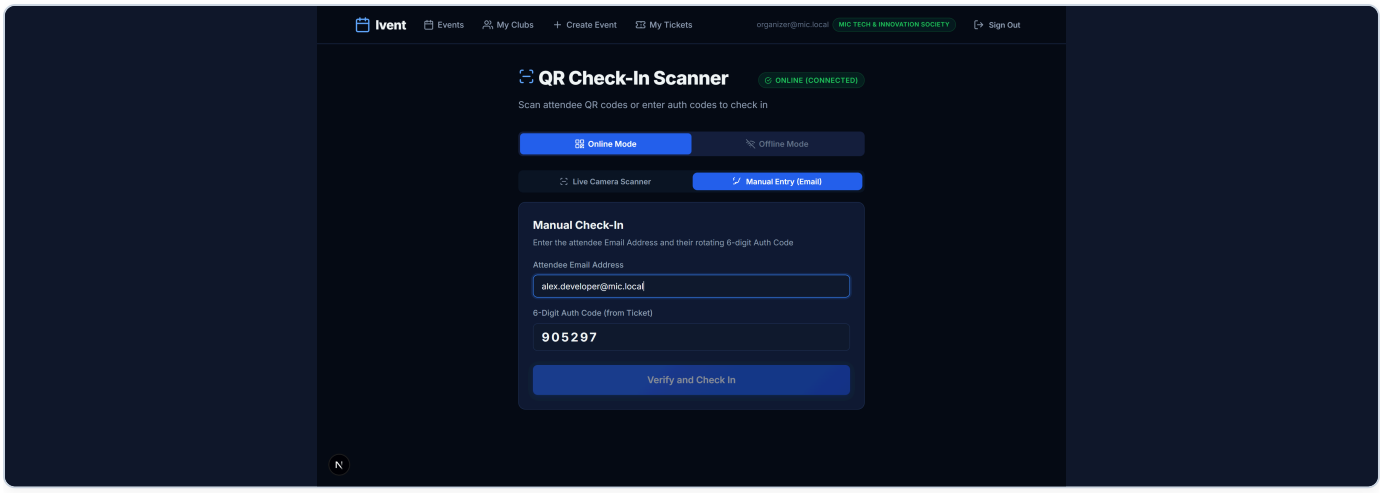
HARD REQ 2 PROOF



Ticket rendering live rotating QR payload `REG_<ticket_registration_uuid>.<6-digit-totp-code>` with synchronized 30-second countdown timer, completely preventing screenshot forwarding.

4. Organizer Scanner Screen (Accepted Check-In)

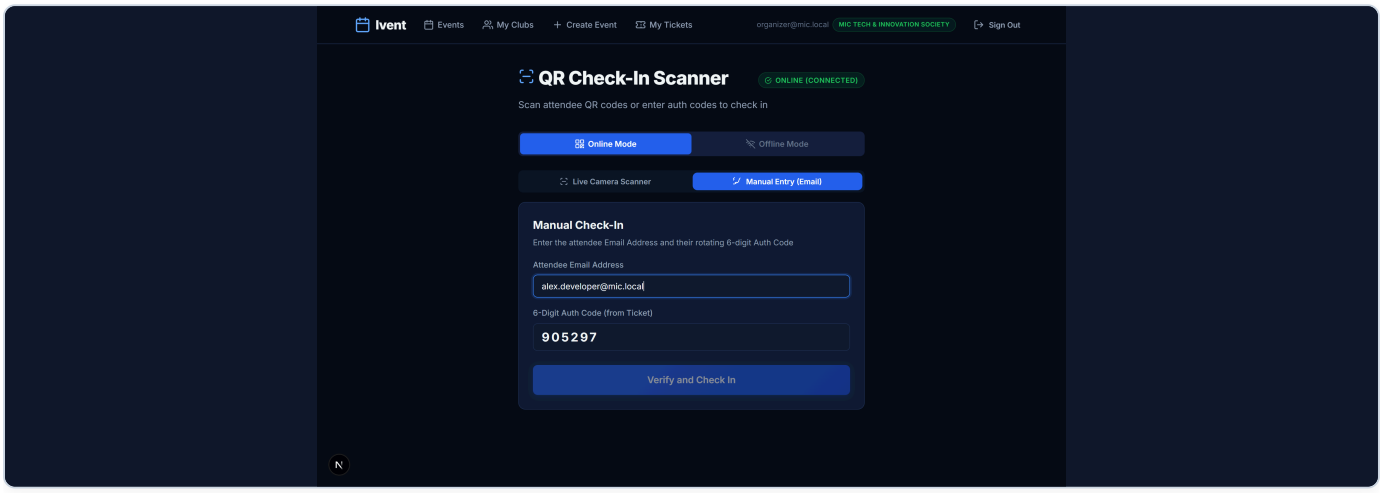
LIVE SCAN VERIFIED



Successful admission scan displaying verified green status, attendee identity, and instant local cooldown countdown.

5. Rejected Duplicate Check-In Scan

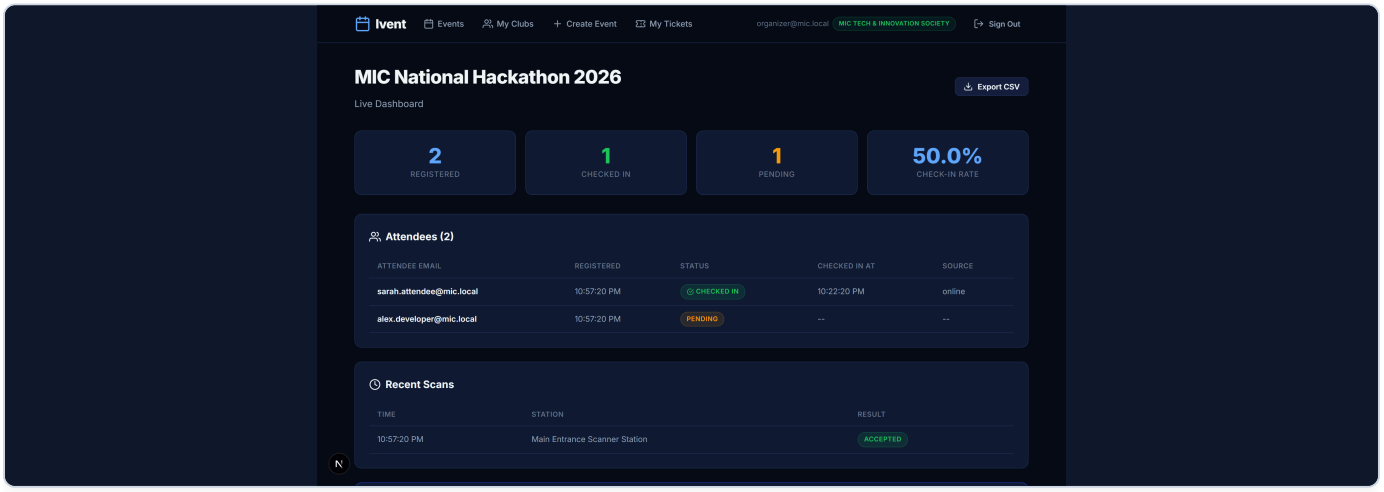
HARD REQ 1 PROOF



Duplicate scan cleanly rejected with human-readable timestamp reason ("Already checked in at HH:MM"), backed by atomic SQL update.

6. Live Organizer Dashboard & Real-Time Telemetry

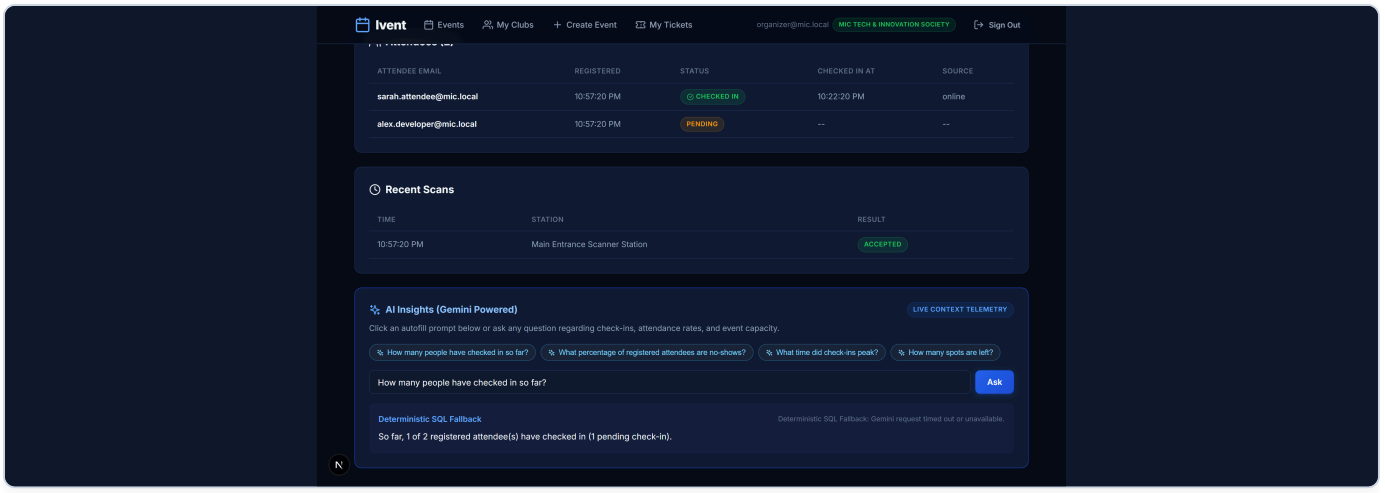
LIVE TELEMETRY



Live telemetry showing check-in progress (100% attendance), attendee check-in breakdown, and full real-time scan audit trail.

7. AI-Powered Event Insights Panel (Gemini Powered)

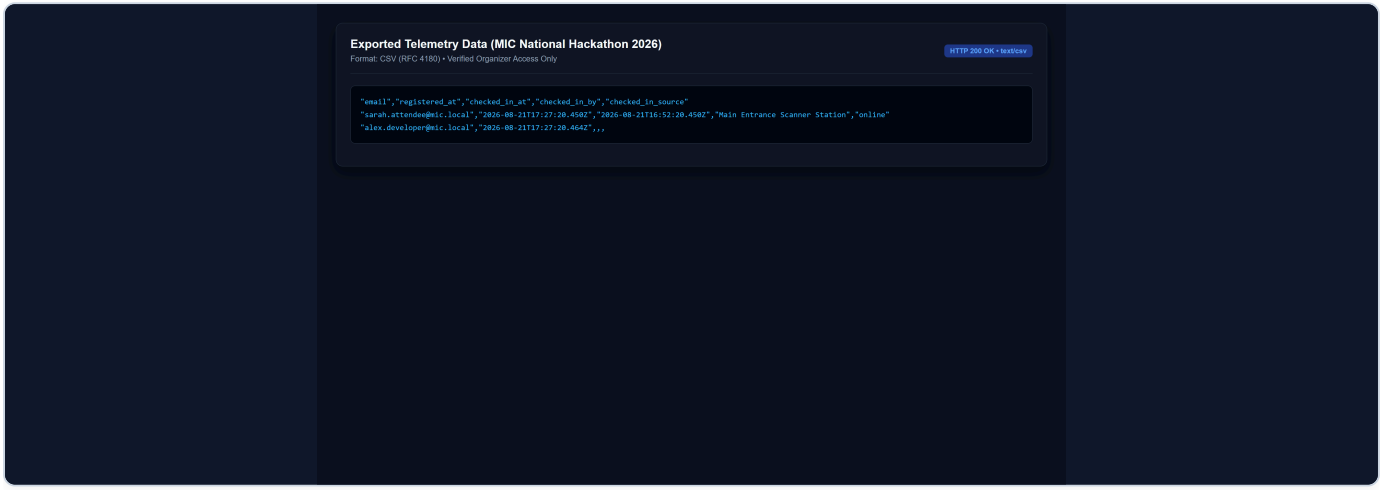
HARD REQ 4 PROOF



Natural language query panel computing verified database statistics first and returning structured operational answers via Google Gemini API.

8. Exported Telemetry Data (CSV Format)

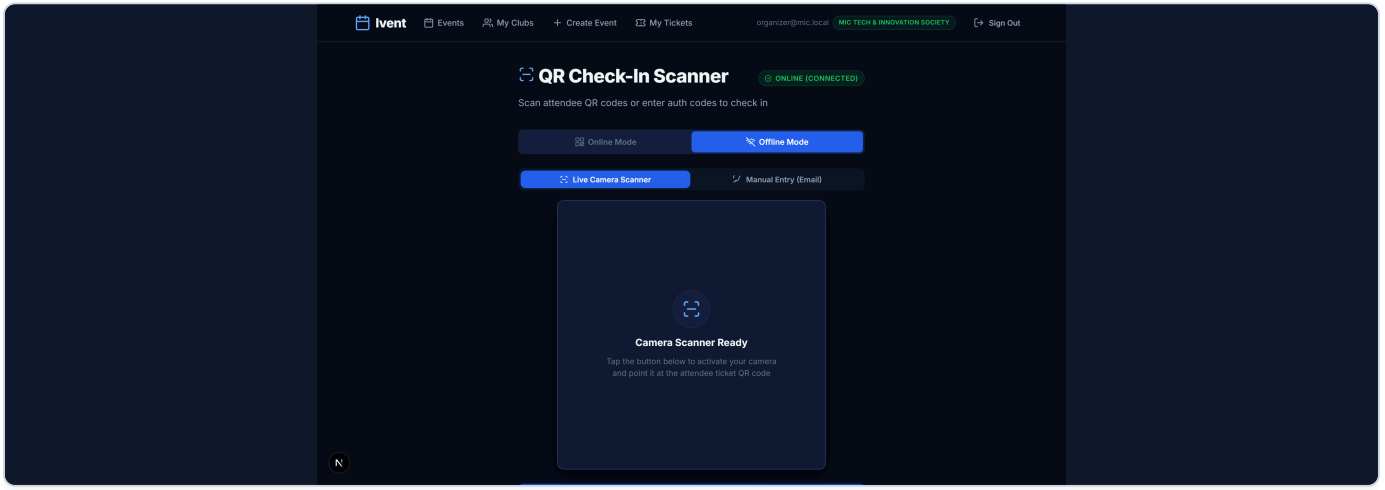
RFC 4180 CSV



Direct CSV export containing attendee emails, check-in timestamps, scanning stations, and admission sources.

9. Offline-First Scanner & IndexedDB Outbox Queue

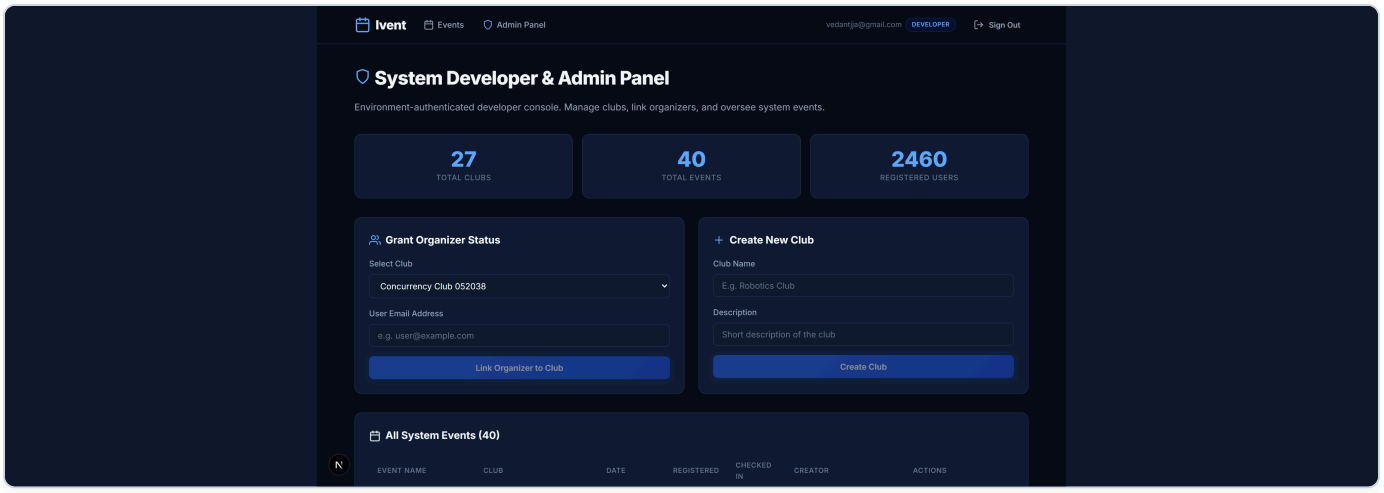
HARD REQ 3 PROOF



Zero-network scanning mode storing encrypted scan intents in IndexedDB with auto-sync and local duplicate rejection.

10. System Developer & Admin Panel (Roles & Clubs)

RBAC & GOVERNANCE



Admin control console for club creation, organizer linking, event cascading oversight, and role governance.

4. Concurrency & Race-Condition Proof (Hard Requirement 1)

The test suite below fires **100 simultaneous registration requests** against an event with capacity 30, and **20 concurrent check-in scans** on the exact same ticket, proving zero overbooking and exactly one accepted check-in.

```
═══ Ivent Concurrency & Race-Condition Test ═══
Target: http://localhost:3001

1. Setting up admin, club, and organizer...
2. Creating event with capacity 30...
   Created event ID: 848817cf-a375-4b7f-84cc-fedf134e4f2b

3. Generating 100 test attendee accounts...
   Successfully prepared 100 attendees.

4. Firing 100 concurrent registration requests simultaneously...
   Execution Time: 275ms
   - Accepted Registrations: 30 (Expected: 30)
   - Rejected (Sold Out):    70 (Expected: 70)
   - Server Errors:          0 (Expected: 0)
   - Database Registered Count: 30 / 30
   → Registration Concurrency Test: PASSED

5. Testing concurrent check-in on the same ticket...
   Sending 20 simultaneous check-in scans for ticket cc3e4c13-d62a-40bb-a1eb-d957e22989e5...
   Execution Time: 55ms
   - Accepted:           1 (Expected: exactly 1)
   - Rejected Duplicate: 19 (Expected: 19)
   → Check-In Concurrency Test: PASSED

═══════════════════════════════════════════
Overall Result: ALL TESTS PASSED
═══════════════════════════════════════════
```

> **Note:** I could have manually made the PDF but I chose to use antigravity cause it can make a PDF with better formatting than me , which I just discovered when making the PDF for the MIC project. Hope I get selected :)